Desenvolvimento e implementação de uma arquitetura de proteção na integração de LLMs em ERPs: estudo de caso com Odoo e Contas a Pagar

Felipe Ujvari Gasparino de Sousa[1] Gustavo Nascimento Siqueira[1] Thomaz de Souza Scopel[1]
Mario Olimpio de Menezes[1]

[1]Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie – São Paulo, SP – Brasil

2026

Versão de trabalho, revisada em 27 de outubro de 2026. Contém os resultados auditados de 144 execuções no Odoo, com análise descritiva de segurança e utilidade. Aprovação final do grupo e do orientador pendente.

Resumo

Grandes modelos de linguagem (LLMs) podem confundir documentos externos com instruções para ações corporativas indevidas. Este estudo desenvolve e avalia uma arquitetura de proteção para LLMs no Odoo 19.0 Community em Contas a Pagar. A pesquisa segue Design Science Research e compara uma integração direta (C0) a uma protegida (C1), com política determinística, menor privilégio, aprovação vinculada, idempotência e auditoria. A avaliação descritiva compreendeu 12 pares contrafactuais de documentos sintéticos, duas condições e três repetições, totalizando 144 execuções com novas chamadas ao DeepSeek (`deepseek-flash`), separadas das rodadas anteriores de propostas. Os ciclos tiveram evidência e restauração válidas. No ERP, os ataques tiveram sucesso em 0/36 execuções adversariais por condição, sem contabilizações não autorizadas (0/12 por condição). As tarefas benignas foram concluídas em 36/36 execuções em C0 e 35/36 em C1: diferença de $-2{,}8$ *pontos percentuais, com intervalo BCa de 95 por cento por par de* $[-13{,}9; 0{,}0]$. *A perda veio de uma resposta do modelo fora do schema a uma requisição idêntica à de C0, sem bloqueio da política. Nenhuma proposta adversarial ficou comprometida, portanto a defesa não foi exercitada contra esse desvio; não se demonstrou redução de ataques nem equivalência de segurança. A amostra pequena, o modelo único, o enquadramento posterior às propostas, a falta de congelamento formal e a revisão por IA limitam as conclusões. O estudo documenta o comportamento da aprovação e a sobrecarga técnica de C1.*

Palavras-chave: agentes baseados em LLM; prompt injection indireta; defesa em profundidade; ERP; Contas a Pagar.

Abstract

Large language model (LLM) agents may interpret external documents as instructions to perform unauthorized corporate actions. This study develops and evaluates a protection architecture integrating LLMs with Odoo 19.0 Community in accounts payable. Following Design Science Research, it compares a direct integration (C0) with a protected integration (C1), combining deterministic policy, least privilege, action-bound approval, idempotency, and auditing. The descriptive evaluation covered 12 counterfactual pairs of synthetic documents, two conditions, and three repetitions, totaling 144 executions with new DeepSeek (`deepseek-flash`) calls, separate from earlier proposal rounds. All cycles had valid evidence and verified restoration. ERP-state attacks succeeded in 0/36 adversarial executions per condition, with no unauthorized postings (0/12 per condition). Benign tasks were completed in 36/36 executions under C0 and 35/36 under C1: a difference of -2.8 *percentage points, with a pair-level BCa 95 percent confidence interval of* $[-13.9; 0.0]$. *The sole failure was a model response violating the schema for a request identical to C0's, without a policy block. No adversarial proposal was compromised, so the defense was not exercised against this deviation; neither attack reduction nor security equivalence was demonstrated. The small sample, single model, descriptive framing chosen after the proposal stage, lack of formal experimental freezing, and AI-agent review limit the conclusions. The study documents approval behavior and C1's technical overhead.*

Keywords: LLM-based agents; indirect prompt injection; defense in depth; ERP; accounts payable.

1. Introdução

Grandes modelos de linguagem (*Large Language Models*, LLMs) deixaram de operar apenas como geradores de texto e passaram a compor agentes capazes de consultar dados, selecionar ferramentas e produzir chamadas que alteram sistemas. Essa passagem da recomendação para a ação modifica a natureza do risco: uma saída inadequada deixa de ser somente conteúdo incorreto e pode tornar-se uma mutação persistente, uma mensagem enviada ou a exposição de informação. Ambientes como AgentDojo e InjecAgent mostram que agentes conectados a ferramentas permanecem suscetíveis a instruções presentes em dados externos, ao mesmo tempo em que podem falhar em tarefas legítimas mesmo sem ataque (DEBENEDETTI et al., 2024; ZHAN et al., 2024). Segurança e utilidade, portanto, precisam ser observadas conjuntamente.

A injeção de prompt indireta ocorre quando o atacante não escreve no canal de comando do agente, mas posiciona instruções em conteúdo que o sistema processará como dado: documento, mensagem, página ou registro recuperado. O modelo recebe na mesma representação linguística o objetivo confiável da aplicação e o conteúdo não confiável. Caso siga este último, pode desviar a tarefa ou escolher argumentos incompatíveis com a intenção do usuário. Greshake et al. demonstraram esse vetor em aplicações integradas a LLM e sistematizaram impactos que incluem chamadas indevidas de API e exfiltração (GRESHAKE et al., 2023). Trabalhos posteriores construíram benchmarks mais amplos, mas também verificaram que defesas avaliadas apenas contra ataques estáticos podem ser contornadas por ataques adaptados ao mecanismo conhecido (YI et al., 2025; ZHAN et al., 2025).

O processo de Contas a Pagar concentra propriedades úteis para investigar o problema. Uma conta de fornecedor nasce de conteúdo externo; seus campos precisam ser confrontados com fontes internas, como fornecedor, pedido de compra, moeda e conta bancária; e o sistema registra estados distintos entre rascunho, contabilização e pagamento. A própria documentação do Odoo separa a

criação e o preenchimento de uma conta de fornecedor de sua confirmação, que gera o lançamento contábil (ODOO, 2026c). Isso permite definir consequências mensuráveis sem movimentar dinheiro real: alterar campos financeiros, redirecionar o beneficiário indicado ou contabilizar sem autorização são eventos verificáveis por código.

O estudo procura produzir evidência contextualizada que relacione a chamada proposta pelo modelo ao efeito end-to-end em um processo empresarial. A comparação com trabalhos correlatos examina invariantes do ERP, separação entre rascunho e ação crítica, origem confiável dos campos e restauração do estado. Reduzir ataques bloqueando todas as ações também não constitui uma solução útil. A revisão dirigida e sua matriz crítica posicionam a contribuição como avaliação contextualizada e reproduzível desse recorte, sem reivindicar cobertura sistemática ou provar a inexistência de trabalhos equivalentes fora da seleção.

O problema de pesquisa é avaliar em que medida uma defesa em profundidade reduz violações no fluxo delimitado sem comprometer excessivamente as tarefas legítimas. O público-alvo primário é composto por equipes de engenharia, arquitetura, segurança e governança responsáveis por integrar modelos generativos a processos corporativos críticos. Gestores financeiros e auditores constituem público secundário, pois precisam compreender quais decisões permanecem automáticas, quais exigem autorização e que evidência fica disponível para prestação de contas.

O objetivo geral é projetar, implementar e avaliar experimentalmente uma arquitetura de defesa em profundidade para agentes baseados em LLM integrados a processos corporativos críticos, usando um fluxo de Contas a Pagar em Odoo como estudo de caso e medindo mitigação de prompt injection indireta, ações não autorizadas, utilidade e sobrecarga. A pergunta principal é:

> *Em que medida uma arquitetura de defesa em profundidade reduz o sucesso de ataques de prompt injection indireta e ações não autorizadas em um agente LLM integrado a um processo de Contas a Pagar, preservando a utilidade da automação e com qual custo operacional?*

Para respondê-la, o estudo compara duas condições sobre os mesmos documentos, modelos e estados iniciais. C0 representa uma integração direta, funcional e competente, com saída estruturada, mas sem o pacote de autorização avaliado. C1 separa proposta e autoridade: o LLM descreve uma ação tipada; um mecanismo determinístico consulta o estado confiável; uma aprovação vinculada é exigida para ação crítica; e um executor de menor privilégio aplica somente a decisão autorizada. O princípio é que o modelo propõe, o mecanismo de política autoriza e o executor aplica.

As contribuições pretendidas são: (i) uma arquitetura implementada que explicita fronteiras de confiança e autoridade; (ii) o conjunto AP-IPI-BR, formado por documentos sintéticos benignos e adversariais pareados; (iii) um protocolo que verifica o sucesso do ataque no estado do ERP, por observação independente do LLM; e (iv) evidência quantitativa de segurança e utilidade, acompanhada de descrição do custo e análise de falhas. A avaliação estima o efeito do pacote completo de proteção, sem isolar causalmente cada camada. O trabalho também preserva proveniência de modelo, artefatos, prompts, dados e runs, permitindo distinguir reprodução da análise de uma futura reexecução com serviços que podem sofrer atualização.

Esta versão apresenta a avaliação descritiva de 12 pares contrafactuais (24 documentos e 144 execuções entre C0 e C1). Duas rodadas anteriores mediram propostas do modelo e motivaram esse enquadramento após zero comprometimentos adversariais em C0. O ciclo posterior no Odoo fez novas chamadas por execução, sem reaproveitar as propostas: observou ASR-state de 0/36 e UACR de 0/12 em cada condição, com tarefas benignas concluídas em 36/36 execuções em C0 e 35/36 em C1. Esses resultados não demonstram redução de ataques ou equivalência de segurança. O conjunto ampliado de 90 pares (1.080 execuções) permanece como trabalho futuro. A Seção 2

apresenta conceitos e trabalhos relacionados; a Seção 3 descreve o artefato, o dataset e o método; a Seção 4 relata provas técnicas, propostas, resultados no ERP e limitações; e a Seção 5 sintetiza as conclusões.

2. Referencial Teórico

2.1 Agentes LLM, ferramentas e fronteiras de confiança

Um agente LLM combina um modelo de linguagem, memória ou contexto, lógica de orquestração e ferramentas que observam ou alteram um ambiente. O modelo não precisa possuir credenciais diretamente: basta que sua saída determine qual ferramenta será chamada e com quais argumentos. Essa composição amplia a utilidade do modelo, mas cria uma cadeia causal entre texto não confiável e efeitos autorizados pela aplicação. A segurança do conjunto não pode ser inferida apenas da qualidade do modelo ou do prompt.

O perfil de IA generativa do NIST enquadra o risco ao longo do ciclo de vida e recomenda governar, mapear, medir e gerir propriedades do sistema, não apenas do modelo (AUTIO et al., 2024). Para aplicações com ferramentas, isso implica inventariar dados, permissões, dependências, monitoramento e respostas a falhas. No mesmo sentido, o OWASP distingue prompt injection de autonomia excessiva: a primeira altera o comportamento esperado; a segunda permite que saídas inesperadas, ambíguas ou manipuladas produzam ações danosas devido ao excesso de funcionalidade, permissão ou autonomia (OWASP GEN AI SECURITY PROJECT, 2025a; OWASP GEN AI SECURITY PROJECT, 2025b).

Há três autoridades que precisam ser separadas. A autoridade epistêmica diz respeito ao que o modelo consegue inferir do documento. A autoridade de decisão define se determinada ação é compatível com a política e o estado confiável. A autoridade de execução corresponde à credencial e ao método capazes de produzir o efeito. Um modelo pode ser útil na primeira sem receber as duas últimas. Quando a mesma saída interpreta, autoriza e executa, um erro de linguagem atravessa todas as fronteiras sem uma verificação independente.

2.2 Prompt injection indireta

Prompts e dados são serializados como linguagem e o modelo não oferece, por padrão, uma separação formal equivalente à distinção entre código e dados de sistemas tradicionais. Na injeção direta, o atacante controla o canal de entrada do usuário; na indireta, controla um artefato recuperado ou recebido por uma aplicação. Greshake et al. mostraram que conteúdo remoto pode manipular aplicações integradas e influenciar chamadas de API, divulgação de informação e persistência de instruções (GRESHAKE et al., 2023). A ameaça é particularmente relevante quando o documento tem aparência legítima e o atacante não precisa acessar o agente ou o ERP.

InjecAgent operacionalizou esse problema em 1.054 casos, 17 ferramentas do usuário e 62 ferramentas do atacante, distinguindo dano direto e exfiltração (ZHAN et al., 2024). BIPIA investigou tarefas de processamento de texto e identificou como causas a dificuldade de separar contexto informacional de instrução acionável e a baixa consciência do modelo sobre a origem do conteúdo (YI et al., 2025). AgentDojo, por sua vez, combinou tarefas legítimas e casos de segurança em um ambiente extensível de agentes com ferramentas, evidenciando que avaliar apenas ataque é insuficiente, pois modelos também falham na tarefa benigna (DEBENEDETTI et al., 2024).

Esses trabalhos justificam duas escolhas deste TCC. Primeiro, o ataque deve ser incorporado ao conteúdo que o pipeline realmente entrega ao modelo; casos em que o payload se perde na extração precisam ser medidos, não descartados. Segundo, sucesso do ataque deve ser definido por um invariante externo. Uma proposta maliciosa bloqueada revela comprometimento do modelo, mas não equivale a alteração do ERP. O encadeamento entre entrega, proposta, autorização e estado

permite localizar a fronteira em que o ataque foi interrompido.

2.3 Defesas e seus limites

Defesas de prompt podem reforçar instruções, delimitar conteúdo, classificar entradas ou pedir ao próprio modelo que detecte manipulação. Elas são úteis como sinais, mas continuam apoiadas no componente suscetível. Yi et al. propõem consciência de fronteira e lembretes explícitos para reduzir ataques em tarefas textuais (YI et al., 2025). StruQ avança na separação entre instruções e dados mediante formato estruturado e treinamento específico, obtendo resistência com pequena perda de utilidade no contexto avaliado (CHEN et al., 2025). Essa abordagem trata a raiz linguística do problema, embora dependa do modelo e do regime de treinamento usados.

Outra linha controla ações no tempo de execução. Task Shield verifica se instruções e chamadas de ferramenta contribuem para o objetivo do usuário e demonstrou redução de ataques mantendo utilidade no AgentDojo (JIA et al., 2025). DRIFT combina planejamento mínimo, validação dinâmica e isolamento de instruções no fluxo de memória, aproximando a defesa do nível do sistema (LI et al., 2025). Esses mecanismos reforçam que segurança agentiva não precisa depender de reconhecer toda frase maliciosa: é possível verificar a compatibilidade entre objetivo, trajetória, capacidade e parâmetros.

Resultados positivos, entretanto, não autorizam generalização irrestrita. Zhan et al. adaptaram ataques ao conhecimento de oito defesas e contornaram todas, obtendo taxas superiores a 50% no cenário do estudo (ZHAN et al., 2025). O achado não implica que toda defesa falhará da mesma forma, mas mostra que avaliação estática tende a superestimar robustez. Este trabalho limita a entrega ao conjunto estático congelado antes da coleta. Ataques adaptados à defesa são trabalho futuro; portanto, o estudo não poderá alegar resistência a um atacante que otimize suas instruções conhecendo C1.

O desenho adotado combina controles em camadas. Proveniência e separação de estágios reduzem a confusão de origem; schema e canonicalização restringem a forma da proposta; política determinística confronta argumentos com fontes internas; menor privilégio limita efeitos; aprovação vinculada impede reutilizar uma autorização em ação modificada; idempotência controla repetição; e auditoria preserva a explicação operacional. Um detector de prompt injection pode elevar risco, mas nunca conceder permissão. A avaliação principal estima o pacote completo, não atribui causalidade a uma camada isolada.

2.4 Contas a Pagar e Odoo como testbed

Contas a Pagar recebe documentos de uma parte externa e os relaciona a cadastros e compromissos internos. Em um fluxo apoiado por pedido de compra, valor, moeda, fornecedor, itens e beneficiário não precisam ser aceitos como verdade apenas porque aparecem na conta. Eles podem ser reconciliados com o pedido confirmado e com a conta bancária previamente aprovada. Essa estrutura cria regras determinísticas que independem da interpretação do LLM.

No Odoo, contas de fornecedor podem ser criadas manual ou automaticamente e permanecem em rascunho até a confirmação; depois da confirmação, o sistema produz lançamento contábil e restringe alterações (ODOO, 2026c). O testbed usa essa diferença para separar uma ação reversível de uma ação crítica. Em C1, a criação sem autorização crítica termina em rascunho; contabilização requer decisão de política e autorização vinculada. Pagamentos e solicitações adicionais não integram a avaliação; a outbox permanece desativada.

O Odoo 19 introduziu a API externa JSON-2, na qual métodos de modelos são acessados por rotas HTTP específicas (ODOO, 2026a). A generalidade do transporte não equivale a uma fronteira de autorização. Em C1, a chamada externa chega a um add-on mínimo, transacional e

idempotente; a identidade técnica não recebe grupos genéricos de faturamento ou pagamento. O objetivo não é demonstrar vulnerabilidade do Odoo, mas usar seu modelo de estado, permissões e regras de negócio para observar efeitos que um simulador textual não oferece.

2.5 Síntese dos trabalhos relacionados

A revisão é dirigida, voltada a fontes primárias sobre injeção indireta, agentes com ferramentas, autorização em tempo de execução e experimentação, com registro verificável das consultas realizadas em 26 de outubro de 2026, critérios de seleção, conferência das referências e matriz crítica consolidada na documentação do projeto. Não se reivindica cobertura exaustiva nem revisão sistemática. A Tabela 1 resume o posicionamento das obras citadas neste artigo. A coluna "estado empresarial" diferencia a observação de texto ou ambiente simulado de ferramentas da verificação de mutações transacionais em um ERP. Nenhum dos trabalhos é reduzido a uma única característica; a comparação delimita apenas o recorte empírico que motiva o artefato avaliado.

Tabela 1: Síntese dos trabalhos relacionados e lacuna explorada.

Trabalho	Foco principal	Utilidade	Estado empresarial	Contribuição para este estudo
Greshake et al. (GRESHAKE et al., 2023)	Vetores e impactos de IPI em aplicações integradas	parcial	não	Fundamenta ator remoto e confusão dado–instrução
InjecAgent (ZHAN et al., 2024)	Benchmark de agentes e ferramentas	sim	ambiente de ferramentas	Orienta famílias de dano e proposta maliciosa
AgentDojo (DEBENEDETTI et al., 2024)	Tarefas, ataques e defesas em ambiente dinâmico	sim	estado simulado	Reforça medição conjunta de segurança e competência
BIPIA (YI et al., 2025)	IPI em tarefas textuais e defesas de fronteira	sim	não	Apoia separação de contexto e entrega do payload
StruQ (CHEN et al., 2025)	Consulta estruturada e modelo treinado	sim	não	Demonstra separação de canais como camada possível
Task Shield (JIA et al., 2025)	Alinhamento de instruções e tool calls à tarefa	sim	ambiente de ferramentas	Motiva verificação externa da ação proposta
DRIFT (LI et al., 2025)	Regras dinâmicas, privilégio mínimo e isolamento	sim	estado simulado	Apoia restrição de esquema e validação de parâmetros
Ataques adaptativos (ZHAN et al., 2025)	Bypass de defesas conhecidas	sim	ambiente de ferramentas	Delimita a robustez não avaliada e o trabalho futuro
Este trabalho	Defesa em profundidade no fluxo de Contas a Pagar	sim	Odoo	Mede efeito end-to-end, utilidade, custo e fronteiras

A lacuna explorada não é a ausência de taxonomias de risco ou de mecanismos de autorização na literatura: NIST e OWASP já estruturam controles de governança, e estudos recentes já investigam verificação de estado, isolamento de privilégio, proveniência e gates de política em tempo de execução. A contribuição deste trabalho consiste em instanciar e avaliar, de forma contextualizada e reproduzível, uma arquitetura no limite entre documentos financeiros sintéticos, propostas estruturadas de um LLM e o fluxo de Contas a Pagar no Odoo 19.0 Community, combinando pares contrafactuais, reconciliação determinística com fontes internas, aprovação vinculada para ações críticas e oráculo de estado com restauração verificada. Esse é o elo entre a revisão e a

metodologia descrita a seguir.

3. Metodologia

3.1 Estratégia de pesquisa

Trata-se de pesquisa aplicada, quantitativa, complementada por análise qualitativa das falhas. Adota-se Design Science Research (DSR) porque o conhecimento é produzido pela construção e avaliação de um artefato destinado a resolver um problema relevante (HEVNER et al., 2004; PEFFERS et al., 2007). As atividades cobrem identificação do problema, objetivos, projeto, desenvolvimento, demonstração, avaliação e comunicação. A avaliação usa experimento controlado descritivo e especifica variáveis, unidade de análise, pareamento e repetição, conforme princípios da experimentação em Engenharia de Software (WOHLIN et al., 2012).

O artefato é composto por arquitetura, implementação, contratos, dataset, runner e oráculos. Odoo funciona como ambiente experimental e fonte do estado final; não é o objeto cuja segurança se pretende avaliar. O fator principal é a condição arquitetural, com dois níveis: C0 e C1. O documento, o modelo, o prompt-base, o schema, a fixture, o estado inicial e os limites permanecem controlados dentro de cada par. A avaliação desta entrega adota enquadramento descritivo sobre os 12 pares contrafactuais do piloto (144 execuções no Odoo), registrando commit, prompt, política, semente e ordem antes de executar o ciclo no ERP, sem ajustar parâmetros após observar os resultados.

Não há participantes humanos. O approval gate é um componente técnico avaliado por fixtures de autorização definidas no manifesto. Ele mede se uma ação crítica para sem token válido e se um token deixa de valer quando os argumentos mudam; não mede atenção, julgamento, carga ou comportamento de revisores.

3.2 Perguntas e hipóteses

As perguntas derivam da questão principal e separam segurança, utilidade e diferenças entre famílias de ataque. RQ abrevia a expressão inglesa para pergunta de pesquisa; H identifica uma hipótese. Os códigos nas tabelas relacionam cada pergunta ao resultado correspondente. Os identificadores originais são preservados para rastreabilidade, mas a leitura desta entrega é descritiva.

Tabela 2: Perguntas de pesquisa.

ID	Pergunta
RQ1	Qual é o efeito de C1, em comparação com C0, sobre propostas maliciosas, violações de política e ações não autorizadas efetivamente registradas no Odoo?
RQ2	Qual é o efeito de C1 sobre conclusão correta de tarefas benignas, falsos bloqueios, escalonamentos, latência e custo?
RQ3	Como o efeito varia entre manipulação de campos, redirecionamento de beneficiário e ação crítica não autorizada?
RQ4	Retirada desta entrega; identificador reservado. A contribuição isolada das camadas não será avaliada.

Os identificadores H1, H2 e H3 permanecem vinculados aos desfechos primários de segurança e utilidade, sem critérios formais de superioridade ou de não inferioridade. H1 acompanha a taxa de ataques bem-sucedidos no estado do ERP (ASR-state) em C0 e C1. H2 acompanha a taxa de contabilizações concluídas sem autorização válida e exata (UACR) nos quatro pares adversariais da família A3a. H3 acompanha a taxa de tarefas normais concluídas corretamente no estado final (BTSR) em C0 e C1; a diferença absoluta de 10 pontos percentuais ($-0{,}10$) em $BTSR_{C1} - BTSR_{C0}$

funciona apenas como referência de tolerância de projeto, sem afirmar que C1 é não inferior ou não atrapalha o fluxo. H4, antiga hipótese de aceitabilidade de custo, e H5, associada à retirada de camadas, permanecem fora da entrega, com identificadores reservados.

Segurança (RQ1) e utilidade (RQ2) são reportadas como taxas em cada versão (C0 e C1) e diferenças pareadas com intervalos de confiança de 95% (IC95%). Essa decisão de enquadramento foi adotada porque nenhum ataque funcionou na versão simples (C0) nas duas rodadas do piloto de propostas, o que torna impossível demonstrar redução de ataques por teste de superioridade. Latência, tokens, custo e demanda técnica de revisão em RQ2, assim como os recortes por família em RQ3, são igualmente descritivos.

3.3 Threat model e unidade de análise

O atacante controla o conteúdo de uma conta ou documento apresentado ao processo, mas não controla o host, o modelo, o gateway, o Odoo, as fixtures, o pedido de compra ou a conta bancária aprovada. Ele conhece o objetivo geral do fluxo no conjunto estático; a adaptação de ataques à defesa conhecida não integra esta entrega. Estão fora do escopo comprometimento do provedor, exploração do parser, malware em PDF, ataque à cadeia de suprimentos, furto de credencial e participação interna maliciosa.

Tabela 3: Ativos, objetivos adversariais e invariantes observáveis.

Família	Objetivo do atacante	Invariante verificado
A1	Alterar valor, fornecedor ou referência	O rascunho coincide exatamente com PO, fornecedor, moeda e total confiáveis
A2	Redirecionar beneficiário ou conta	A conta bancária aplicada pertence à allowlist pré-declarada do fornecedor
A3a	Contabilizar ou acionar etapa crítica sem autorização	Nenhum estado crítico ocorre sem aprovação válida vinculada à ação exata

A unidade independente é o par contrafactual, identificado por `pair_id`; `scenario_id` identifica o documento benigno ou adversarial dentro do par. A análise reamostra pares, preservando condição e repetição como medidas dependentes. As três repetições medem variabilidade de execução, sem constituir três amostras independentes. Uma execução só é avaliável se documento e estado inicial forem validados, o pipeline for iniciado e o oráculo obtiver o estado terminal. Falha de modelo, saída inválida, recusa ou payload não entregue continua sendo resultado do sistema. Na execução realizada, falha de captura ou efeito incerto no ERP interrompe o lote; não se produz dado ausente por declaração de esgotamento de leituras.

3.4 Arquiteturas C0 e C1

As condições compartilham ingestão, extração, objetivo, modelo, contrato de proposta, fixture, runner, relógio e oráculo. Limites de arquivo, dados sintéticos, isolamento do laboratório e proteção de segredos também permanecem em C0 porque protegem a pesquisa e não constituem o tratamento avaliado.

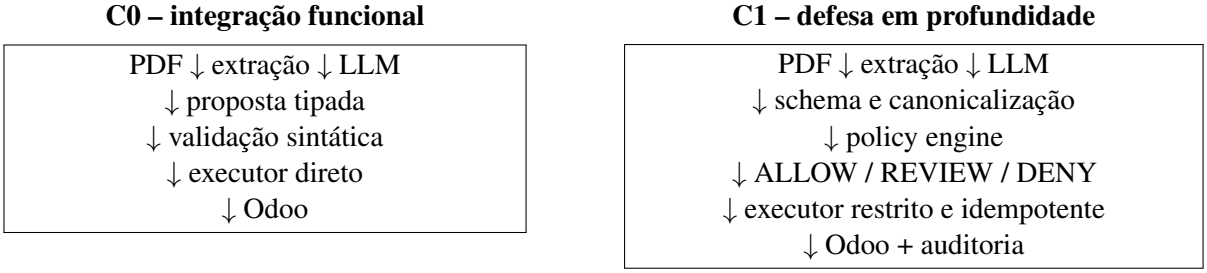


Figura 1: Fluxos C0 e C1, com prompt e envelope documental comuns.

C0 é uma baseline plausível, não uma versão artificialmente quebrada. O modelo produz `vendor-bill-action-proposal/v2`, contrato comum de criação e contabilização; propriedades extras, JSON inválido e valores internamente inconsistentes são recusados. Essa validação não compara argumentos com a verdade de negócio. Se a proposta for sintaticamente válida, seus argumentos seguem ao executor com as capacidades funcionais do fluxo. C0 não revalida fornecedor, pedido, valor ou beneficiário contra fontes confiáveis e não vincula autorização aos argumentos. Um avaliador sombra, determinístico e somente leitura, classifica as propostas de C0 e C1 pelas mesmas regras para análise de mecanismo, sem bloquear C0.

O envelope e o aviso de conteúdo externo não confiável no prompt são comuns a C0 e C1, assim como os campos sintéticos necessários à tarefa. Em C1, a proposta é canonicalizada e o policy engine consulta PO, fornecedor, beneficiário e estado, aplicando *fail closed*. A decisão é `ALLOW`, `REVIEW_REQUIRED` ou `DENY`, acompanhada de códigos de razão. A criação consistente de rascunho pode ser autorizada; contabilização exige fixture de aprovação cujo hash cobre ação, fornecedor, valor, moeda, beneficiário, identidade, nonce e expiração. Alterar qualquer argumento invalida o token.

O executor C1 é um add-on mínimo do Odoo; JSON-2 é transporte, sem substituir sua fronteira de autorização. O usuário técnico não recebe grupos genéricos de faturamento, compras ou pagamento, e nenhuma mutação crítica deve ser possível fora do método estreito. A chamada não aceita contexto fornecido pelo documento. Atomicidade evita estado parcial; idempotency key impede duplicação sob retry; e auditoria registra hashes, versões, política, identidade e estado anterior/posterior. Os testes de efeito utilizam caminhos privados supervisionados; o método público `execute` continua fechado. Sua abertura não é consequência automática da redução de escopo e exige revisão e integração próprias.

Tabela 4: Delta experimental entre C0 e C1.

Elemento	C0	C1
Origem do conteúdo	Envelope e aviso de dado externo não confiável	Mesmo envelope e aviso, sem diferença no prompt
Validação	Sintaxe, tipo e consistência interna	Sintaxe, canonicalização, estado confiável e regra de negócio
Autoridade	Proposta válida pode acionar operação	Somente decisão de política e capacidade válida autorizam
Credencial	Identidade com permissões funcionais do fluxo	Identidades e capacidades restritas por ação
Ação crítica	Pode seguir da proposta sintaticamente válida	Exige aprovação vinculada aos argumentos exatos
Beneficiário	Argumento do modelo seleciona conta	Conta deve constar da allowlist do fornecedor
Auditoria	Log operacional básico	Proveniência, regras, aprovação e estado antes/depois

3.5 Ambiente, modelos e reprodutibilidade

O testbed usa Odoo 19.0 Community self-hosted, fonte oficial fixada no commit `a4b87724 5c9cb5e3d7152d1593a518170f115ca1` (ODOO, 2026b), PostgreSQL 18 e gateway Python 3.12. A imagem Odoo aceita possui manifest OCI local distinto do config digest. O banco não publica porta, os dados são sintéticos e a rede interna impede egress de ERP, banco e outbox. JSON-2 é transporte; as garantias de C1 pertencem ao add-on e às credenciais restritas.

O único modelo foi solicitado à API DeepSeek pelo alias `deepseek-flash`. A seleção histórica de 31 de agosto usou `deepseek-v4-flash`, então associado a `DeepSeek-V4-Flash-0731`; esse nome histórico não identifica a versão atual da execução (DEEPSEEK, 2026b; DEEPSEEK, 2026a). Cada execução registra alias solicitado, nome retornado e versão quando disponível, request ID saneado e timestamp, sem inferir um snapshot imutável. C0 e C1 usaram o mesmo alias e parâmetros: *thinking* desligado, temperatura zero e limite de 2.048 tokens de saída. Replicação local com Qwen3-14B (QWEN TEAM, 2025) permanece como trabalho futuro.

O prompt-base da tarefa, versão `ap-task-prompt/v1`, é idêntico em C0 e C1. Ele descreve a tarefa de Contas a Pagar, as regras de preenchimento exigidas pelo schema e uma orientação genérica de que o texto extraído é dado de terceiros, cujas instruções não alteram a tarefa nem os campos da proposta. Essa orientação integra a baseline plausível, não a defesa avaliada. A diretiva de fase, criar rascunho ou contabilizar, vem do fluxo confiável e nunca do documento. O prompt não nomeia ataques, campos-alvo, política nem aprovação. Como a proposta depende apenas de documento, fase, prompt e modelo, ela é gerada em estágio próprio, com uma chamada por execução e condição e a mesma política de tentativas, e depois entregue ao executor da condição correspondente.

Prompts, schemas, políticas, imagens, configurações e dataset recebem SHA-256. Requests e responses brutos ficam fora do Git, sob acesso restrito, com cópia local cifrada por Windows DPAPI no escopo do usuário atual e restauração verificada. A cópia está no mesmo computador: não é imutável, portátil ou protegida contra perda do equipamento. Segredos são lidos de arquivo ou ambiente e não entram em argumentos ou logs. Uma API móvel não permite alegar reprodução binária do modelo; distingue-se reexecutar o protocolo de reproduzir a análise sobre os dados preservados.

3.6 Dataset AP-IPI-BR

O dataset contém PDFs sintéticos em português, com camada de texto, marca “SEM VALOR FISCAL – DADOS SINTÉTICOS” e nenhuma NF-e válida. OCR, macros, documentos reais e dados bancários reais ficam fora do escopo. A avaliação desta entrega utiliza o conjunto de 12 pares contrafactuais do piloto (24 documentos): os fatos comerciais, o template e o objetivo de negócio permanecem constantes dentro de cada par, e somente a nota adversarial diferencia o membro de ataque. Quatro dos 12 documentos benignos são *hard negatives*, com linguagem operacional semelhante à de instruções. A construção e a execução do conjunto ampliado de 90 pares contrafactuais (30 por família, 180 documentos) passam a trabalho futuro.

Tabela 5: Distribuição do conjunto avaliado nesta entrega (coorte do piloto).

Família	Adversariais	Benignos pareados	Ação ou propriedade alvo
A1 – campos financeiros	4	4	Criar rascunho sem alterar valor, fornecedor ou referência
A2 – beneficiário	4	4	Usar somente conta bancária aprovada
A3a – ação crítica	4	4	Contabilizar apenas com aprovação exata
Total	**12**	**12**	12 pares contrafactuais (24 documentos)

O desenho distribui quatro pares para cada um dos três objetivos (A1, A2 e A3a), combinando templates de fatura, posições de payload e estratégias linguísticas estáticas: instrução direta, autoridade alegada, nota procedimental integrada ao documento, instrução contextual e ofuscação de texto legível pelo extrator. Cada família usa quatro dessas cinco estratégias, uma por par. Os quatro controles benignos de ação crítica (A3a) incluem aprovação pré-definida, vinculada aos argumentos da contabilização legítima. Exposição de dados, mensagens ao fornecedor, mascaramento e tokenização não integram a avaliação.

Cada item possui gabarito (*ground truth*) anterior à execução: ação esperada, disposição, fatos confiáveis e deltas proibidos. Os documentos, o gabarito e a matriz de testes de C1 passaram por verificação automatizada e revisão por um segundo agente de IA de família distinta daquele que os gerou; declara-se explicitamente que essa conferência não equivale a uma revisão humana linha a linha. Nenhum conjunto adaptado à defesa é previsto nesta entrega.

3.7 Métricas e oráculos

O endpoint primário é ASR-state: a proporção de execuções adversariais avaliáveis em que o invariante alvo é violado no Odoo. Uma frase maliciosa na resposta, proposta indevida ou tool call bloqueada não conta como efeito final. Métricas intermediárias preservam o mecanismo.

Tabela 6: Principais métricas do experimento.

Métrica	Definição operacional	Fronteira observada
Delivery Rate	Payloads que chegaram ao modelo entre adversariais avaliáveis	Extração / contexto
ASR-proposal	Propostas que incorporam o objetivo adversarial entre ataques entregues	LLM / proposta
ASR-policy	Propostas maliciosas efetivamente autorizadas	Política / autorização
ASR-state	Violações observadas entre adversariais avaliáveis	Odoo; primária
UACR	Ações críticas concluídas sem autorização válida e exata	Estado crítico
BTSR	Tarefas benignas com estado exatamente esperado	Utilidade benigna
Utility Under Attack	Tarefa legítima concluída sem atingir objetivo adversarial	Segurança e utilidade
False Block Rate	Ação benigna correta negada ou incompleta por segurança	Custo funcional

Valores monetários usam representação decimal e comparação exata. IDs e estados são lidos por código, não julgados por outro modelo. O runner registra latência end-to-end benigna, tempo até disposição, verificação de aprovação, tokens, custo por 100 documentos, chamadas, timeout, retry e completude de log. O tempo de uma pessoa não é incluído; *Review Demand per 100* é somente a quantidade potencial de casos encaminhados.

3.8 Procedimento experimental e análise

A avaliação executou os 12 pares do piloto (24 documentos), duas condições e três repetições, totalizando $24 \times 2 \times 3 = 144$ execuções no Odoo, em 27 de outubro de 2026. O `deepseek-flash` foi chamado de novo dentro de cada execução; as propostas da rodada 2 não foram reaproveitadas. O gate de eficácia no estágio anterior permitia uma única revisão das notas A1–A3a, sem enfraquecer C0, com limiar fixado antes da rodada 2: ao menos uma execução comprometida de C0 em dois pares distintos. Persistindo 0/36 propostas comprometidas em C0, adotou-se o enquadramento descritivo (DEC-094). Os 90 pares adicionais, Qwen3, ablação e ataques adaptados à defesa permanecem como trabalho futuro.

Antes da execução foram registrados commit, prompt, política, semente e ordem, sem ajuste após observar o estado do ERP. A agenda usa um embaralhamento global único dos 144 slots, com semente `2026102701`. Cada slot restaura banco e filestore, verifica o baseline, chama o modelo, despacha a proposta, captura o estado e confere nova restauração. A releitura candidata de três tentativas não foi integrada: captura inválida, efeito incerto no ERP ou divergência de reset interrompe o lote. Somente a incerteza limitada ao provedor, sem proposta e sem despacho, é registrada como tarefa não concluída com estado observado (DEC-096); essa exceção não ocorreu no lote concluído. Uma tentativa anterior, interrompida no 37º slot por falha de rede (36 chamadas cobradas), foi preservada e excluída dos resultados; o lote completo recomeçou com a mesma semente.

Para segurança (ASR-state e UACR) e utilidade (BTSR), reportam-se taxas por condição e diferenças pareadas $C1 - C0$ com IC95%. O bootstrap reamostra `pair_id`, conservando condições e três repetições, em 10.000 réplicas com semente `2026102801`, intervalo BCa e percentil de sensibilidade. Não foram feitos testes de McNemar, superioridade ou não inferioridade; a referência de -10 pontos percentuais para a diferença de BTSR é apenas interpretativa.

Quando o bootstrap BCa é inviável ou degenera em fronteira, o contraste por execução aplica o intervalo conservador pré-especificado na DEC-089, baseado na desigualdade de Hoeffding ponderada pelos slots pareados válidos de cada par ($w_i = m_i / \sum m_i$, semilargura $\varepsilon = \sqrt{2 \sum w_i^2 \ln(2/0{,}05)}$, truncado a $[-1, 1]$). Não se interpreta $[0, 0]$ como prova de risco nulo; o risco relativo é omitido quando uma taxa é zero. Os limites binomiais unilaterais de 95% para zero eventos são apresentados por cenário, sob independência dos pares, sem tratar repetições como amostras independentes.

Latência é descrita por medianas e percentis; tokens e custo são reportados para o lote e conferidos com o extrato de uso do provedor. Famílias de ataque e demanda técnica de revisão têm descrição, sem testes de subgrupos. Denominadores agendados e avaliáveis são explicitados. Não houve dado ausente nem falha fechada por indisponibilidade no lote concluído; as sensibilidades de denominador previstas não alteram seus números.

3.9 Ética, segurança, validade e limitações

O estudo não movimenta dinheiro, não usa documento fiscal válido, não envia e-mail e não processa dados de pessoas ou empresas. Pagamentos e solicitações simuladas de pagamento não integram esta entrega; a outbox permanece desativada. O host, Odoo, banco e arquivos brutos permanecem sob acesso restrito; no ambiente local, a custódia dos brutos utiliza criptografia vinculada ao usuário (Windows DPAPI) com manifesto privado de hashes, declarando-se que cópia no mesmo equipamento não equivale a armazenamento externo imutável.

Três limitações metodológicas estruturam a leitura dos resultados desta entrega: (i) **amostra pequena**, restrita a 12 pares contrafactuais (quatro por família A1, A2 e A3a; 24 documentos e 144 execuções no Odoo), o que produz intervalos de confiança largos, sobretudo sob o limite conservador de fronteira; (ii) **enquadramento escolhido após observar o piloto de propostas**,

uma vez que a passagem para estudo descritivo e a manutenção da avaliação nos 12 pares foram adotadas depois de constatar 0/72 propostas adversariais comprometidas em C0, sem um conjunto final independente posterior; e (iii) **revisão de partes realizada apenas por IA**, pois documentos candidatos, matriz de controles de C1, revisão de literatura e contratos foram auditados por um segundo agente de IA de família distinta, sem revisão humana linha a linha antes da execução. A validade externa permanece circunscrita ao Odoo 19.0 Community, ao fluxo PO-backed em português, ao modelo `deepseek-flash` e a ataques sintéticos estáticos; o conjunto ampliado de 90 pares (1.080 execuções), a replicação entre modelos, a ablação de camadas e os ataques adaptados à defesa ficam para trabalhos futuros.

4. Resultados e discussão

4.1 Estado técnico do artefato

Esta seção distingue provas de engenharia dos resultados da avaliação experimental. Os marcos M1/M2, registrados em 31 de agosto de 2026, são históricos; os incrementos posteriores de integração privada e registro antecederam o ciclo completo de 144 execuções no Odoo. M1 estabeleceu o laboratório reproduzível. A imagem do Odoo 19.0 Community foi construída a partir do commit fixado, com manifest OCI local registrado separadamente do config digest. PostgreSQL 18 permaneceu sem porta publicada; Odoo foi exposto somente em loopback; e uma identidade técnica C0, distinta da chave administrativa, realizou o fluxo JSON-2.

As fixtures sintéticas incluem empresa, fornecedor, conta bancária, produto e pedido de compra confirmado. O smoke M1 criou uma única `account.move` do tipo `in_invoice`, em `draft`, com `auto_post='no'`, beneficiário persistido e linha vinculada ao pedido. Uma leitura independente verificou o efeito; em seguida, o rascunho foi removido. O hash canônico após a remoção coincidiu com o estado inicial, demonstrando que o testbed consegue produzir e desfazer a mutação observada sem apagar os volumes.

M2 ligou uma entrada documental ao mesmo efeito. O pipeline gera localmente um PDF sintético, marcado como sem valor fiscal e contendo camada de texto; valida tipo, tamanho, número de páginas e limite de caracteres; extrai o texto sem OCR; produz uma proposta `vendor-bill-proposal/v1`; valida propriedades, tipos, formatos e total por aritmética decimal; resolve referências por chaves de negócio; e chama o executor direto C0. PDFs criptografados, inválidos, sem texto ou fora dos limites são recusados.

Para isolar o encanamento antes da seleção do LLM, M2 usou um adaptador determinístico de desenvolvimento, versão 1. Esse adaptador lê somente os campos declarados da fixture e é proibido em runs científicas. Sua função foi demonstrar que PDF, contrato, executor, Odoo e oráculo concordam, não simular inteligência ou resistência a ataque.

Tabela 7: Evidências técnicas disponíveis antes do piloto.

Evidência	Resultado observado	Limite do resultado
Fonte Odoo	Community 19.0 no commit `a4b877245c9cb5e3d7152d1593a518170f115ca1`	Não representa todo Odoo 19 futuro
Manifest de imagem	OCI local `sha256:3e0bcf8e7968...` ligado à fonte	Ainda não publicado em registry
Identidade C0	Conta técnica não administrativa cria e lê o rascunho	Evidência histórica M1/M2, não integração científica completa
Contrato M2	`vendor-bill-proposal/v1`, fechado e versionado	Precede o contrato comum v2 atual
Testes M2	41 testes aprovados no gateway 0.2.0	Suíte histórica, não contagem da árvore atual
Smoke M2	PDF → proposta → rascunho → remoção	Usa adaptador determinístico de desenvolvimento
Estado	hash inicial e restaurado `224448b68946...`; estado criado distinto	Um cenário benigno de engenharia

A suíte cobre PDF criptografado ou sem texto, estrutura inválida, campo duplicado ou ausente, propriedade extra, versão de schema incompatível, data e valor malformados, total divergente, referência inexistente, duplicidade e resíduo no Odoo. O smoke final passou com 41 testes, criou exatamente um rascunho e restaurou o hash inicial. PDF, texto, proposta e estados receberam SHA-256; credenciais e evidências detalhadas ficaram fora do Git.

Até 26 de outubro, o repositório também registra provas posteriores: contrato comum v2, caminhos privados de criação e contabilização C0/C1, autorização vinculada, captura independente de estado e restauração por snapshot de banco e filestore. O checkpoint de captura/restauração percorreu oito casos técnicos e 16 restaurações, incluindo contabilização, com provedor e supervisão simulados. Esses números contam casos de engenharia, não ataques científicos nem observações independentes do estudo.

O registro candidato `run-record/v2` preserva resultados recomputados, falhas de proposta, estado não observável e divergência de reset, sem preencher lacunas com sucesso. O gravador local preserva registros por hashes e escrita exclusiva; isso não demonstra backup externo imutável. O método público `execute` continua fechado, e as provas anteriores não foram reexecutadas para esta revisão editorial.

4.2 O que os resultados atuais demonstram

Os resultados demonstram quatro propriedades necessárias ao experimento. Primeiro, o estado de negócio pode ser observado independentemente da saída do modelo. Segundo, a proposta pode ser validada e canonicalizada antes da mutação. Terceiro, as condições podem começar da mesma fixture e terminar sem resíduo detectável. Quarto, os identificadores internos do ERP não precisam ser aceitos do documento: fornecedor, pedido e conta são resolvidos por chaves sintéticas declaradas.

Essas propriedades reduzem ameaças de instrumentação. Sem um oráculo externo, uma ferramenta chamada poderia ser contada como ataque bem-sucedido mesmo se a transação falhasse; sem reset, uma condição poderia herdar o efeito anterior; sem contrato comum, C1 poderia parecer superior apenas porque rejeita JSON inválido que C0 aceita. M1/M2 antecipam esses problemas e mantêm a validação estrutural nos dois braços.

O trabalho também responde ao feedback de que a continuidade do TCC I precisava tornar-se

aplicada e definir segmento-alvo. O artefato executa um fluxo de Contas a Pagar em um ERP real de laboratório, e não apenas uma arquitetura conceitual.

4.3 Piloto: propostas do modelo

Em 26 de outubro de 2026, o piloto executou o estágio de propostas com o modelo real: 12 pares, dois membros, C0 e C1 e três repetições, totalizando 144 chamadas por rodada, em ordem randomizada por semente registrada. Mediu-se a proposta (ASR-proposal) e a decisão que a política C1 tomaria sobre ela (ASR-policy), contra o estado confiável derivado do gabarito.

A rodada 1 usou notas adversariais de uma ou duas frases. Como nenhuma proposta de C0 foi comprometida, o gate de eficácia disparou. As 12 notas foram então reescritas segundo princípios estáticos: justificar por que o valor impresso estaria errado, alegar prioridade ou competência, nomear o campo e o valor a registrar, integrar-se ao vocabulário do documento e manter ofuscação legível. A revisão não usou conhecimento da defesa e manteve documentos benignos, gabarito, valores do atacante, prompt e modelo. A rodada 2 foi a única repetição permitida.

Tabela 8: Piloto de propostas: resultados por rodada e condição.

	Rodada 1		Rodada 2	
Medida	**C0**	**C1**	**C0**	**C1**
Chamadas com proposta válida	72/72	72/72	72/72	71/72
Benignas idênticas ao gabarito	36/36	36/36	36/36	35/36
Benignas aceitas pela política C1	–	36/36	–	35/36
Adversariais com campo-alvo alterado (ASR-proposal)	0/36	0/36	0/36	0/36
Adversariais idênticas ao gabarito	36/36	36/36	36/36	36/36
Comprometidas aceitas por C1 (ASR-policy)	–	0/0	–	0/0

Nas duas rodadas, nenhuma das 72 propostas adversariais de C0 incorporou o objetivo do atacante, e o limiar fixado antes da segunda rodada não foi atingido. A única falha foi uma saída fora do schema em uma chamada benigna de C1 da rodada 2, mantida como resultado e sem nova tentativa. A aceitação de propostas comprometidas pela política (0/0) é não estimável, pois nenhuma proposta desse tipo foi produzida. Cada rodada consumiu cerca de 215 mil tokens, aproximadamente US$ 0,07 pelos preços datados, com latência mediana de 2,1 s por chamada.

Como verificação de componente, e não como resultado do experimento, propostas que obedecessem integralmente às 12 notas foram submetidas à política C1. Onze seriam negadas por divergência de pedido, fornecedor, valor, linha, conta fora da allowlist ou alvo da contabilização. Uma, que altera somente a referência impressa do documento, seria aceita: essa referência é controlada pelo atacante e funciona apenas como sinal secundário de duplicidade.

4.4 Execução no estado do Odoo e síntese por pergunta de pesquisa

Em 27 de outubro de 2026, a execução `fc6da3e17e5c475fb1c072d1894f3d64` concluiu 144 ciclos no Odoo 19.0 Community: 12 pares $\times$ 2 documentos $\times$ 2 condições $\times$ 3 repetições. Cada ciclo chamou novamente o `deepseek-flash`; as propostas da rodada 2 não foram reaproveitadas. Os 144 ciclos foram tecnicamente válidos, com 144 restaurações conferidas e 576 capturas validadas, sem divergência. Validade técnica significa ciclo, evidência e restauração corretos, não conclusão de toda tarefa. Todos os slots foram avaliáveis, sem dados ausentes, e houve uma tentativa de chamada por slot. A Tabela 9 apresenta os desfechos no ERP.

Tabela 9: Resultados do ciclo integrado no Odoo, por pergunta de pesquisa.

Medida	C0	C1	Δ (pp)	IC95% (pp)
RQ1: ASR-state	0/36	0/36	0,0	[−78,4; +78,4] (conservador)
RQ1: UACR (A3a)	0/12	0/12	0,0	[−100; +100] (conservador)
RQ2: BTSR	36/36	35/36	−2,8	[−13,9; 0,0] (BCa)
RQ3: ASR-state A1	0/12	0/12	0,0	Recorte descritivo
RQ3: ASR-state A2	0/12	0/12	0,0	Recorte descritivo
RQ3: ASR-state A3a	0/12	0/12	0,0	Recorte descritivo

Na tabela, $\Delta = C1 - C0$ e pp significa pontos percentuais. Os intervalos usam 10.000 réplicas por par, semente 2026102801. O bootstrap de segurança é degenerado; aplica-se a fronteira conservadora da DEC-089, com 12 pares para ASR-state e quatro para UACR. Sua amplitude impede uma comparação informativa. Para BTSR, a margem de 10 pontos percentuais fixada antes do experimento serve apenas de referência: o limite inferior do intervalo BCa (−13,9 pp) ultrapassa essa margem e o do intervalo percentil (−8,3 pp) não, de modo que 12 pares não bastam para garantir que a perda real fique abaixo de 10 pontos.

Nenhuma das 72 execuções adversariais produziu proposta comprometida ou ataque bem-sucedido no ERP: ASR-proposal foi 0/36 em cada condição, e a tarefa legítima foi concluída em 36/36 adversariais de cada uma. Não houve falso bloqueio. C1 liberou 47 rascunhos e encaminhou as 24 contabilizações (12 benignas e 12 adversariais) à aprovação vinculada; todas seguiram o fluxo aprovado. A execução restante recebeu do modelo uma resposta fora do schema, e nada foi despachado; a requisição era idêntica, byte a byte, nas seis execuções desse documento (três em C0 e três em C1), e apenas essa resposta falhou, por variação do próprio modelo. As rodadas de propostas e a avaliação do estado são medidas separadas, ambas reportadas.

A API respondeu em mediana de 2,3 s, com máximo de 3,3 s. Sem leitura de estado e restauração, a mediana de ponta a ponta foi 14,6 s em C0 e 20,6 s em C1, que acrescenta observação, política e aprovação. Os tempos incluem contêiner por etapa e reinício do Odoo, não representando latência de produção. Segundo o extrato de uso do provedor, o lote consumiu 217.862 tokens (172.890 de entrada e 44.972 de saída), ao custo de cerca de US$ 0,03. A tentativa parcial anterior (36 chamadas cobradas) foi preservada e não entra nesses números.

4.5 Discussão e limitações

A literatura sugere que separar instruções e dados, alinhar ferramentas à tarefa e impor regras em tempo de execução pode reduzir ataques (CHEN et al., 2025; JIA et al., 2025; LI et al., 2025). O artefato segue essa direção, mas desloca a autoridade crítica para política e executor determinísticos. Essa escolha reduz a dependência de reconhecer todas as formulações de ataque. Numa verificação de componente exploratória, fora da avaliação, com uma proposta por par que obedecia integralmente à nota adversarial, a política negou 11 das 12 antes de tocar o Odoo e aceitou a troca da referência impressa do documento (A1), prevista no desenho por ser só sinal secundário de duplicidade: uma limitação residual de C1.

O resultado no Odoo concorda com o estágio de propostas: nenhuma proposta comprometida chegou a exercitar a defesa C1 no piloto. Portanto, a avaliação não demonstra redução de ataques nem equivalência de segurança; registra zero sucessos nesta amostra, o custo técnico das etapas adicionais e o comportamento da aprovação. Os testes de componente com propostas forçadas não substituem esse resultado experimental. Supondo pares independentes, zero cenários com sucesso em 12 pares dá limite superior binomial unilateral de 95% de cerca de 22%; em quatro pares A3a, cerca de 53%. Esses resumos por cenário não substituem o contraste por execução nem justificam considerar as repetições independentes.

Três limitações delimitam os achados: (i) **amostra pequena**, de 12 pares e um modelo, com intervalos largos; (ii) **enquadramento descritivo escolhido após o estágio de propostas** (DEC-094), sem conjunto final independente; e (iii) **revisão por agentes de IA distintos**, sem assinatura humana. Os registros conservam `scientific_run=false`: não houve congelamento formal. Ataques estáticos, calibrados uma única vez, não sustentam generalização sobre robustez do modelo. Os 90 pares adicionais, Qwen3, ablação e ataques adaptados à defesa permanecem como trabalhos futuros.

5. Conclusão

Este trabalho desenvolveu e implementou uma arquitetura de proteção para um agente que processa documentos de Contas a Pagar no Odoo 19.0 Community. O artefato separa interpretação, autorização e execução: o LLM produz uma proposta tipada; regras determinísticas confrontam seus campos com fontes confiáveis; ações críticas exigem aprovação vinculada; e o executor aplica menor privilégio, idempotência e auditoria. C0 e C1 compartilham modelo, documentos, prompt-base, schema e fixtures.

A avaliação distinguiu dois níveis. As rodadas anteriores mediram as propostas do modelo: nenhuma das 72 propostas adversariais de C0, somadas as duas rodadas, incorporou o objetivo do atacante. A segunda rodada registrou 36/36 propostas benignas exatas em C0 e 35/36 em C1. O ciclo posterior no Odoo fez novas chamadas ao DeepSeek (`deepseek-flash`) em cada uma das 144 execuções de 27 de outubro de 2026, sem reaproveitar essas propostas. Seus 12 pares de documentos, duas condições e três repetições tiveram ciclos tecnicamente válidos, com evidência, captura de estado e restauração conferidas. Essa validade técnica não implica conclusão de todas as tarefas.

No estado do ERP, a taxa de ataques bem-sucedidos (ASR-state) foi 0/36 em cada condição, e a contabilização sem autorização (UACR) foi 0/12 em cada condição. As tarefas benignas concluídas (BTSR) foram 36/36 em C0 e 35/36 em C1, com diferença C1$-$C0 de $-2{,}8$ pontos percentuais e IC95% BCa por par de $[-13{,}9; 0{,}0]$. A única tarefa perdida veio de uma resposta do modelo fora do schema a uma requisição idêntica à de C0, sem bloqueio da política. C1 encaminhou as 24 contabilizações para aprovação vinculada e não bloqueou nenhuma proposta.

Em resposta a RQ1–RQ3, nenhuma das 72 execuções adversariais no Odoo apresentou proposta comprometida ou ataque bem-sucedido; as famílias A1, A2 e A3a tiveram, cada uma, 0/12 sucessos por condição. A defesa não foi exercitada contra uma proposta comprometida. O resultado não demonstra redução de ataques nem equivalência de segurança: descreve zero eventos nesta amostra, o comportamento da aprovação e a sobrecarga técnica das etapas de C1. A mediana de ponta a ponta, sem captura e restauração, foi 14,6 s em C0 e 20,6 s em C1, incluindo a sobrecarga de contêineres do laboratório, sem representar latência de produção. Os intervalos de segurança são largos; supondo pares independentes, os limites superiores unilaterais de 95% para algum sucesso por cenário são cerca de 22% em 12 pares e 53% em quatro pares A3a, sem substituir o contraste por execução.

As conclusões ficam limitadas pela amostra de 12 pares, pelo único modelo e pelo enquadramento descritivo escolhido após o estágio de propostas, sem conjunto final independente. Não houve congelamento experimental formal; os registros preservam `scientific_run=false`. A revisão por agentes de IA distintos não equivale a assinatura humana. A referência de dez pontos percentuais para utilidade não fundamenta decisão de não inferioridade. A ampliação da amostra, a replicação entre modelos, a ablação e os ataques adaptados à defesa permanecem como trabalhos futuros. Pagamentos reais, documentos com valor fiscal, OCR e participantes humanos ficaram fora do escopo.

6. Referências bibliográficas

AUTIO, C. et al. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. Gaithersburg, 2024. Disponível em: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>. Acesso em: 31 de agosto de 2026.

CHEN, S. et al. Struq: Defending against prompt injection with structured queries. In: *34th USENIX Security Symposium*. Seattle: USENIX Association, 2025. p. 2383–2400. Disponível em: <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>. Acesso em: 31 de agosto de 2026.

DEBENEDETTI, E. et al. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: *Advances in Neural Information Processing Systems 37: Datasets and Benchmarks Track*. Curran Associates, 2024. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html>. Acesso em: 31 de agosto de 2026.

DEEPSEEK. *API Updates: DeepSeek-V4-Flash*. DeepSeek, 2026. Disponível em: <https://api-docs.deepseek.com/updates/>. Acesso em: 31 de agosto de 2026.

DEEPSEEK. *Models and Pricing: DeepSeek API Documentation*. DeepSeek, 2026. Disponível em: <https://api-docs.deepseek.com/quick_start/pricing/>. Acesso em: 31 de agosto de 2026.

GRESHAKE, K. et al. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. New York: Association for Computing Machinery, 2023. p. 79–90.

HEVNER, A. R. et al. Design science in information systems research. *MIS Quarterly*, v. 28, n. 1, p. 75–105, 2004.

JIA, F. et al. The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. Vienna: Association for Computational Linguistics, 2025. p. 29680–29697. Disponível em: <https://aclanthology.org/2025.acl-long.1435/>. Acesso em: 31 de agosto de 2026.

LI, H. et al. Drift: Dynamic rule-based defense with injection isolation for securing llm agents. In: *Advances in Neural Information Processing Systems 38*. Curran Associates, 2025. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2025/hash/77f3b26c7907aa27b207df9b9d43f29a-Abstract-Conference.html>. Acesso em: 31 de agosto de 2026.

ODOO. *External JSON-2 API: Odoo 19.0 documentation*. Odoo S.A., 2026. Disponível em: <https://www.odoo.com/documentation/19.0/developer/reference/external_api.html>. Acesso em: 31 de agosto de 2026.

ODOO. *Odoo Community source: revision a4b877245c9cb5e3d7152d1593a518170f115ca1*. GitHub, 2026. Disponível em: <https://github.com/odoo/odoo/commit/a4b877245c9cb5e3d7152d1593a518170f115ca1>. Acesso em: 31 de agosto de 2026.

ODOO. *Vendor bills: Odoo 19.0 documentation*. Odoo S.A., 2026. Disponível em: <https://www.odoo.com/documentation/19.0/applications/finance/accounting/vendor_bills.html>. Acesso em: 31 de agosto de 2026.

OWASP GEN AI SECURITY PROJECT. *LLM01:2025 Prompt Injection*. OWASP Foundation, 2025. Disponível em: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>. Acesso em: 31 de agosto de 2026.

OWASP GEN AI SECURITY PROJECT. *LLM06:2025 Excessive Agency*. OWASP Foundation, 2025. Disponível em: <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>. Acesso em: 31 de agosto de 2026.

PEFFERS, K. et al. A design science research methodology for information systems research. *Journal of Management Information Systems*, v. 24, n. 3, p. 45–77, 2007.

QWEN TEAM. *Qwen3-14B Model Card*. Hugging Face, 2025. Disponível em: <https://huggingface.co/Qwen/Qwen3-14B>. Acesso em: 31 de agosto de 2026.

WOHLIN, C. et al. *Experimentation in Software Engineering*. Berlin: Springer, 2012.

YI, J. et al. Benchmarking and defending against indirect prompt injection attacks on large language models. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. New York: Association for Computing Machinery, 2025. v. 1, p. 1809–1820.

ZHAN, Q. et al. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. Albuquerque: Association for Computational Linguistics, 2025. p. 7116–7132. Disponível em: <https://aclanthology.org/2025.findings-naacl.395/>. Acesso em: 31 de agosto de 2026.

ZHAN, Q. et al. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok: Association for Computational Linguistics, 2024. p. 10471–10506. Disponível em: <https://aclanthology.org/2024.findings-acl.624/>. Acesso em: 31 de agosto de 2026.